

Name: Atchaya Vharsne S

Reg. No.: 192524185

Course Code: CSA0401

Course Name: Operating Systems

CO3 AT2

1. Logical Address and Physical Address

A logical address is the address generated by the CPU when a program is running. It is also called a virtual address. The CPU does not directly access the actual location in RAM using this address.

A physical address is the actual address of a location in the main memory (RAM). The memory unit uses the physical address to access the required instruction or data.

Address Translation

The process of converting a logical address into a physical address is called address translation.

The basic process is:

CPU → Logical Address → Address Translation → Physical Address → Main Memory

For example, suppose a process generates the logical address 250. If the process is loaded into memory starting at physical address 1000, then:

Physical Address = Base Address + Logical Address

= 1000 + 250

= 1250

Therefore, the CPU-generated logical address 250 is translated into physical address 1250.

Logical addresses allow programs to work independently of their actual location in memory. The operating system and hardware perform the translation before memory access.

Difference

Logical Address	Physical Address
Generated by CPU	Actual RAM address
Also called virtual address	Actual memory address
Used by programs	Used by memory hardware
Requires translation	Does not require further translation

Thus, address translation provides memory flexibility, protection, and efficient memory management.

2. Address Translation Using Base and Limit Registers

The base and limit registers are hardware registers used for address translation and memory protection.

- Base register: Contains the starting physical address of a process.
- Limit register: Contains the maximum valid logical address or size of the process's memory area.

When a process generates a logical address, the system first checks whether the address is within the permitted range.

Working

The steps are:

1. CPU generates a logical address.
2. The logical address is compared with the limit register.
3. If the logical address is greater than or equal to the limit, the access is invalid.
4. If it is valid, the base address is added to the logical address.
5. The resulting value becomes the physical address.

The formula is:

Physical Address = Base + Logical Address

Example

Suppose:

- Base = 5000
- Limit = 2000
- Logical address = 1500

Since:

$1500 < 2000$

the address is valid.

Physical address:

$$5000 + 1500 = 6500$$

Therefore, physical address = 6500.

Now suppose the logical address is 2500.

Since:

$$2500 > 2000$$

the address is invalid. The system generates a memory protection fault/trap and does not allow the process to access that memory.

Importance

Base and limit registers prevent a process from accessing memory belonging to another process or the operating system. Therefore, they provide:

- Memory protection
- Process isolation
- Controlled memory access
- Prevention of unauthorized access

3. Paging

Paging is a memory management technique in which logical memory is divided into fixed-size blocks called pages, while physical memory is divided into fixed-size blocks called frames.

Usually, the size of a page and frame is the same.

For example, if the page size is 4 KB, then:

- Logical memory → Pages of 4 KB
- Physical memory → Frames of 4 KB

A process may have pages such as:

Page 0, Page 1, Page 2, Page 3

Physical memory may have:

Frame 0, Frame 1, Frame 2, Frame 3, ...

The pages do not need to be stored continuously in physical memory.

Example

Suppose:

- Page size = 100 bytes
- Logical address = 250

Page number:

$$250 / 100 = 2$$

Offset:

$$250 \bmod 100 = 50$$

Therefore:

- Page number = 2
- Offset = 50

If page 2 is stored in frame 7:

Physical Address = Frame $\times$ Page Size + Offset

= $7 \times 100 + 50$

= 750

Therefore, logical address 250 corresponds to physical address 750.

Advantages of Paging

Paging eliminates external fragmentation because any available frame can hold any page.

It also improves memory utilization because a process does not need one continuous block of memory.

Disadvantage

Paging can cause internal fragmentation, especially when the last page is not completely filled.

Thus, paging provides flexible and efficient memory allocation.

4. Page Table

A page table is a data structure maintained by the operating system to map logical pages to physical frames.

Each process normally has its own page table.

A logical address is divided into:

Logical Address = Page Number + Offset

The page number is used to find the corresponding frame number in the page table.

Structure

A page table may contain:

Page Number	Frame Number	Valid Bit	Protection
0	5	1	Read/Write
1	2	1	Read Only
2	8	1	Read/Write
3	6	0	—

For example, if the CPU generates:

Page = 2, Offset = 100

The page table shows:

Page 2 → Frame 8

Therefore:

Physical Address = Frame × Page Size + Offset

If page size = 1000 bytes:

= $8 \times 1000 + 100$

= 8100

Advantages

1. Supports non-contiguous memory allocation.
2. Eliminates external fragmentation.
3. Provides memory protection.
4. Allows virtual memory implementation.
5. Makes memory allocation flexible.

Limitations

1. Page tables require additional memory.
2. Large processes require large page tables.
3. Accessing the page table can increase memory access time.
4. Internal fragmentation can occur.
5. TLB or other hardware support is often required for faster translation.

Thus, the page table is an important component of paging because it maintains the relationship between logical pages and physical frames.

5. Translation Lookaside Buffer (TLB)

A Translation Lookaside Buffer (TLB) is a small, fast memory located close to the CPU/MMU. It stores recently used page-table entries.

Its main purpose is to speed up logical-to-physical address translation.

Normally, without a TLB, the system may need to:

1. Access the page table.
2. Find the frame number.
3. Access the actual memory location.

This can require multiple memory accesses.

Working of TLB

Suppose the CPU generates:

Page number = 5, Offset = 100

The system first searches the TLB for page 5.

If page 5 is found

It is a TLB hit.

The TLB immediately provides the frame number.

The system then accesses the required physical memory location.

If page 5 is not found

It is a TLB miss.

The system accesses the page table in main memory, obtains the frame number, and may store the new mapping in the TLB.

Example

Suppose:

- Page 5 $\rightarrow$ Frame 12
- Offset = 50

If the TLB contains:

Page 5 → Frame 12

then:

Physical Address = $12 \times \text{Page Size} + 50$

The translation happens quickly because the page-table entry is already available in the TLB.

Benefits

- Reduces address translation time
- Reduces memory accesses
- Improves CPU performance
- Makes paging faster

Therefore, the TLB acts as a fast cache for page-table entries.

6. TLB Hit and TLB Miss

A TLB hit occurs when the required page number is found in the TLB.

A TLB miss occurs when the required page number is not found in the TLB.

TLB Hit

Suppose the CPU requests page 3.

The TLB contains:

Page 3 → Frame 10

The frame number is obtained immediately.

The system can directly access the required physical memory location.

Therefore, a TLB hit results in lower memory access time.

TLB Miss

Suppose the CPU requests page 7, but page 7 is not present in the TLB.

The system must:

1. Search the page table in main memory.
2. Find the frame corresponding to page 7.
3. Update the TLB.
4. Access the required physical memory location.

Therefore, a TLB miss takes more time than a TLB hit.

Comparison

TLB Hit	TLB Miss
Page entry found in TLB	Page entry not found
Faster	Slower
No page-table lookup normally required	Page-table lookup required
Improves performance	Reduces performance

Example

Suppose TLB lookup takes 10 ns and memory access takes 100 ns.

For a TLB hit, approximately:

$$10 + 100 = 110 \text{ ns}$$

For a TLB miss, the system may need an additional page-table memory access:

$$10 + 100 + 100 = 210 \text{ ns}$$

Thus, a high TLB hit ratio is important for good system performance.

7. Memory Protection Using Base/Limit Registers and Access Permissions

Memory protection prevents one process from accessing memory that belongs to another process or the operating system.

Base/limit registers provide one important method of protection.

- Base register → Starting physical address of the process.
- Limit register → Maximum logical address/size allowed for the process.

Working

Suppose:

- Base = 4000
- Limit = 3000

If the process generates logical address 2000:

Since:

$$2000 < 3000$$

the address is valid.

Physical address:

$$4000 + 2000 = 6000$$

But if the process generates logical address 3500:

$$3500 > 3000$$

Therefore, the access is rejected.

Access Permissions

Memory can also have permissions such as:

- Read
- Write
- Execute

For example, a code segment may be Read + Execute, while certain data areas may be Read + Write.

If a process attempts to write to a read-only area, the hardware generates a protection fault/trap.

Importance in Multi-user Systems

In a multi-user environment, many users and processes share the same computer. Without memory protection:

- One process could modify another process's data.
- A user could access protected operating-system memory.
- Programs could interfere with each other.
- Security could be compromised.

Therefore, memory protection provides security, stability, process isolation, and controlled access.

8. Segmentation

Segmentation is a memory management technique in which a program is divided into logical sections called segments.

Unlike paging, segments are usually variable-sized because they represent logical parts of a program.

Common segments include:

1. Code segment – Contains program instructions.
2. Data segment – Contains global and static data.
3. Stack segment – Contains function calls, local variables, and return addresses.
4. Heap segment – Contains dynamically allocated memory.

A logical address in segmentation consists of:

Segment Number + Offset

Example

Suppose a program has:

Segment	Purpose	Size
0	Code	4000 bytes
1	Data	2000 bytes
2	Stack	3000 bytes

If the CPU generates:

Segment 1, Offset 500

the system looks at segment 1 and checks whether offset 500 is within its limit.

If valid, the offset is added to the segment's base address.

Advantages

- Provides a logical view of programs.
- Supports protection for individual segments.
- Allows sharing of code/data segments.
- Segments can grow independently.
- Matches the programmer's logical view of memory.

Disadvantage

Because segments are variable-sized, segmentation can produce **external fragmentation**.

Thus, segmentation is useful when memory needs to be organized according to the logical structure of a program.

9. Segment Table

A segment table stores information about each segment of a process.

Each segment table entry normally contains:

- Base address – Starting physical address of the segment.
- Limit – Size of the segment.
- Protection information – Read, write, execute permissions.

Example

Segment	Base	Limit
0	5000	1000
1	8000	2000
2	12000	1500

A logical address consists of:

Segment Number + Offset

Suppose the CPU generates:

Segment = 1, Offset = 500

The segment table gives:

- Base = 8000
- Limit = 2000

First, the system checks:

$500 < 2000$

Therefore, the address is valid.

Then:

Physical Address = Base + Offset

$$= 8000 + 500$$

$$= 8500$$

Therefore, the physical address is 8500.

If the offset were 2500, then:

$$2500 > 2000$$

so the address would be invalid and a protection fault would occur.

Translation Process

CPU $\rightarrow$ Segment Number + Offset $\rightarrow$ Segment Table $\rightarrow$ Base + Offset $\rightarrow$ Physical Address

Thus, the segment table performs both address translation and protection.

10. Paging vs Segmentation

Paging and segmentation are both memory management techniques, but they organize memory differently.

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
Physical memory	Fixed-size frames	Variable-size memory areas
Address	Page number + offset	Segment number + offset
Fragmentation	Internal fragmentation	External fragmentation
Programmer view	Usually invisible	Matches logical program structure
Allocation	Pages can be placed in any free frame	Each segment requires suitable memory space
Protection	Page-level protection	Segment-level protection
Sharing	Possible	Easier at logical segment level
Size	Fixed	Variable
Flexibility	Good for physical memory management	Good for logical program organization

Paging

In paging, logical memory is divided into equal-sized pages.

For example:

Page 0 → Frame 4

Page 1 → Frame 8

Page 2 → Frame 1

The pages can be placed anywhere in physical memory.

The major advantage is that external fragmentation is eliminated.

However, the last page may not be completely filled, resulting in internal fragmentation.

Segmentation

In segmentation, the program is divided according to its logical components:

Code | Data | Stack | Heap

Each segment can have a different size.

This makes segmentation more meaningful from a programmer's perspective and allows different permissions for different segments.

However, because segments have variable sizes, external fragmentation can occur.

Performance

Paging usually provides efficient physical memory management, especially when combined with a TLB. Segmentation provides convenient logical organization and protection, but variable-size allocation can make memory management more complicated.